Lume

Apresentação do projeto

O que o Lume resolve, como foi construído e por que cada peça está onde está — em dez páginas, para quem quer conhecer o projeto sem instalar nada.

FICHA DOC-03	004.4 · LUM
ASSUNTO	Problema, solução, arquitetura, decisões e design
PLATAFORMA	Django · Python · Java 21 · PostgreSQL · Redis
PÚBLICO	Recrutadores, colaboradores e curiosos
EDIÇÃO	Versão 1.1.0 · outubro de 2026
LICENÇA	MIT

CREME
#F4F1DE

CAMURÇA
#B0A08C

SÁLVIA
#B8C9A9

ARDÓSIA
#8395A7

ROSÉ
#D8A7B1

Clara A. Portes
em parceria com Erik Pimenta

OUTUBRO DE 2026
RIO DE JANEIRO · PT-BR

01 · EM UMA PÁGINA

Uma biblioteca que lê junto com você

O Lume é uma biblioteca web auto-hospedada para PDF e DOCX. Você solta os arquivos; ele gera as capas, extrai texto e sumário, escreve uma descrição com IA e abre tudo num leitor próprio que lembra onde você parou.

5	3	~13 mil	70	0
SERVIÇOS NO DOCKER COMPOSE	LINGUAGENS PYTHON · JAVA · JS	LINHAS DE CÓDIGO PRÓPRIO	TESTES AUTOMATIZADOS COM PONTA A PONTA	ETAPAS DE BUILD NO FRONT-END

DO ENVIO À LEITURA

01 · ENVIO
Arrastar e soltar em qualquer lugar da janela

02 · FILA
Redis Streams com grupo de consumidores

03 · LEITURA
Processador Java extrai texto, sumário e capa

04 · DESCRIÇÃO
Resumo e etiquetas por IA ou local

05 · LEITOR
Pronto, avisado em tempo real pelo WebSocket

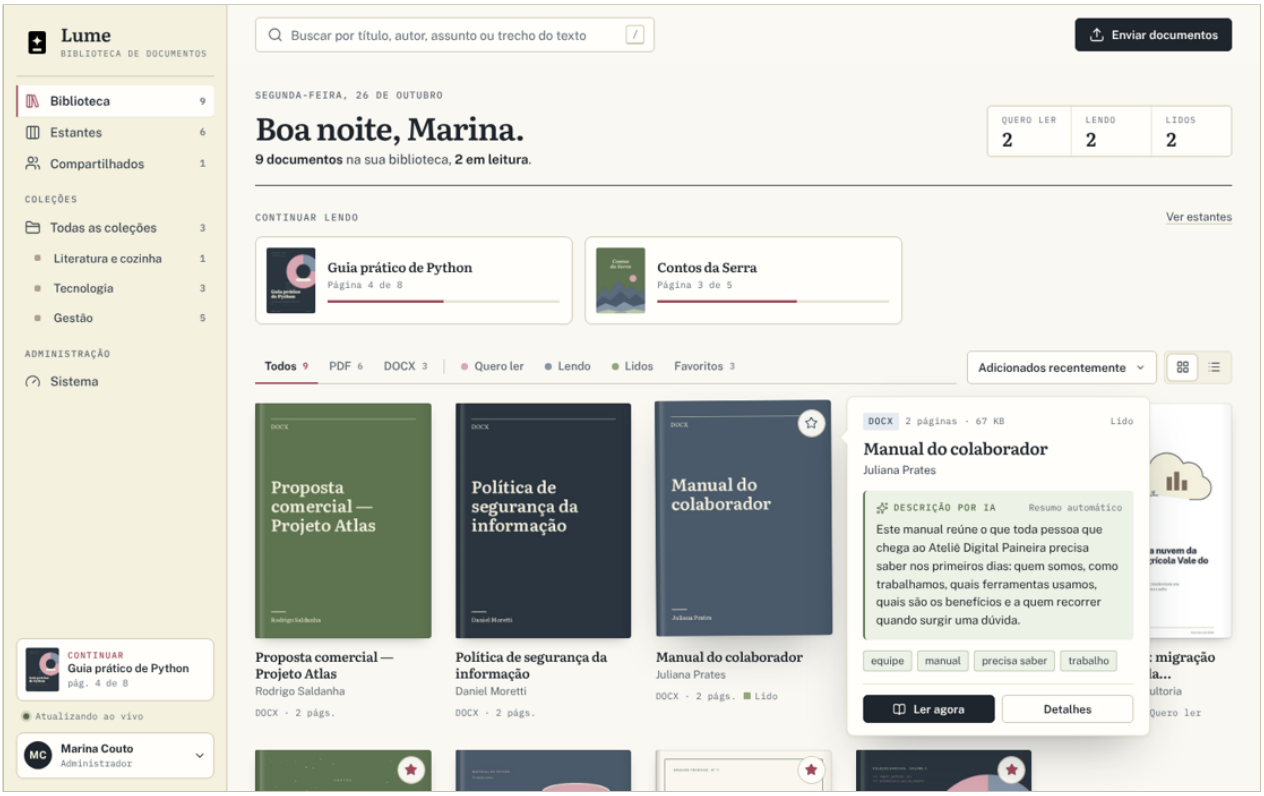


FIG. 1 A biblioteca: cabeçalho com números reais, filtros como abas de índice, capas geradas e a prévia com a descrição por IA.

02 · O PROBLEMA

Documentos espalhados, leitura perdida

Apostilas, manuais, relatórios e atas vivem em pastas, e-mails e aplicativos diferentes — sem capa, sem resumo e sem lembrar onde a leitura parou.

ANTES

- Arquivos com nomes como `final_v3.pdf`, sem pista do conteúdo.
- Para saber do que trata um documento, é preciso abri-lo.
- Busca só pelo nome do arquivo, nunca pelo texto de dentro.
- Cada leitor guarda (ou não) a página e as anotações do seu jeito.
- Compartilhar com a equipe significa mandar o arquivo de novo.

COM O LUME

- Toda entrada ganha capa, autor, páginas e tempo de leitura.
- Duas ou três frases de descrição e etiquetas sugeridas.
- Busca no texto inteiro, sem diferenciar acentos, com o trecho em destaque.
- Posição, destaques, notas e marcadores salvos por pessoa.
- Estantes, coleções e documentos compartilhados com a instalação.

Para quem foi feito

EQUIPES

Um acervo único

Reunir manuais, políticas, contratos e atas num lugar pesquisável, com contas, papéis e compartilhamento.

LEITORES

Uma estante pessoal

Quero ler, lendo e lidos, com progresso, destaques em quatro cores e modo foco — no próprio computador ou servidor.

DESENVOLVEDORES

Um projeto para estudar

Uma aplicação Django com processamento assíncrono, serviço Java e tempo real, toda documentada e testada.

O que entra na biblioteca

RECURSO	COMO FUNCIONA
Capas	A página 1 do PDF vira capa (se estiver em branco, tenta a próxima). DOCX ganha uma capa tipográfica na paleta do projeto.
Descrição por IA	Sem chave, um resumo extrativo local. Com chave, Claude, OpenAI, Ollama ou qualquer API compatível. Se o provedor falhar, o local assume.
Leitor	PDF com pdf.js (zoom, miniaturas, sumário, busca, temas papel, sépia e noite) e DOCX convertido em HTML de leitura.
Organização	Estantes com arrastar e soltar, coleções com ordem própria, favoritos e etiquetas.
Tempo real	Capas, descrições, contadores e estantes se atualizam em todas as abas abertas.
Operação	Painel Sistema com filas, workers, processador e IA; tokens pessoais para a API REST.

03 · O PRODUTO

Feito para ler de verdade

O centro do Lume é o leitor. Tudo o que vem antes — capa, texto, sumário, descrição — existe para que abrir um documento seja rápido e voltar a ele seja natural.

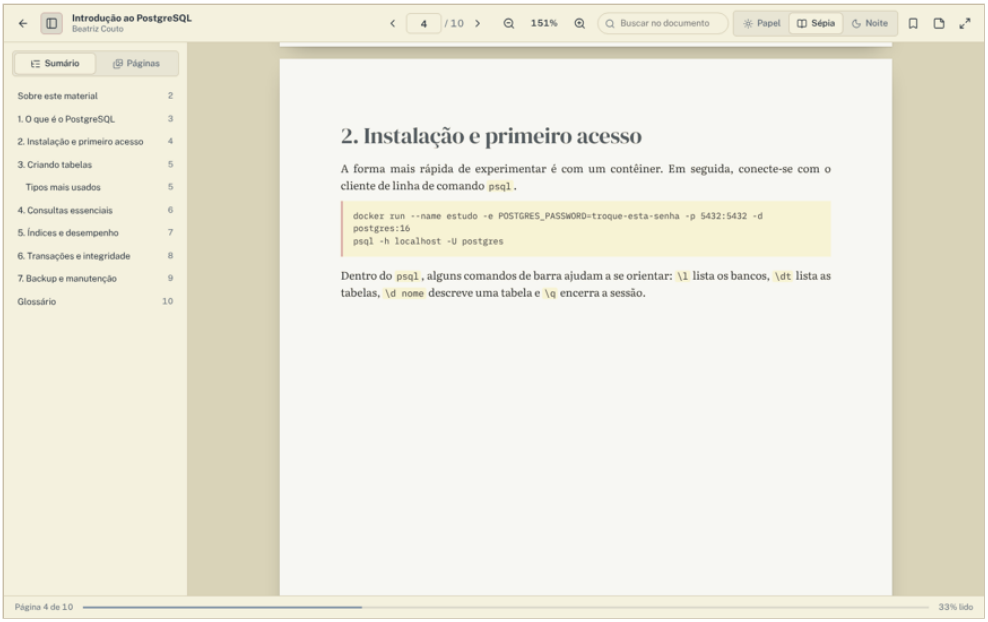


FIG. 2 Leitor de PDF: sumário à esquerda, página aberta, zoom, busca e destaques. O progresso é salvo a cada página.

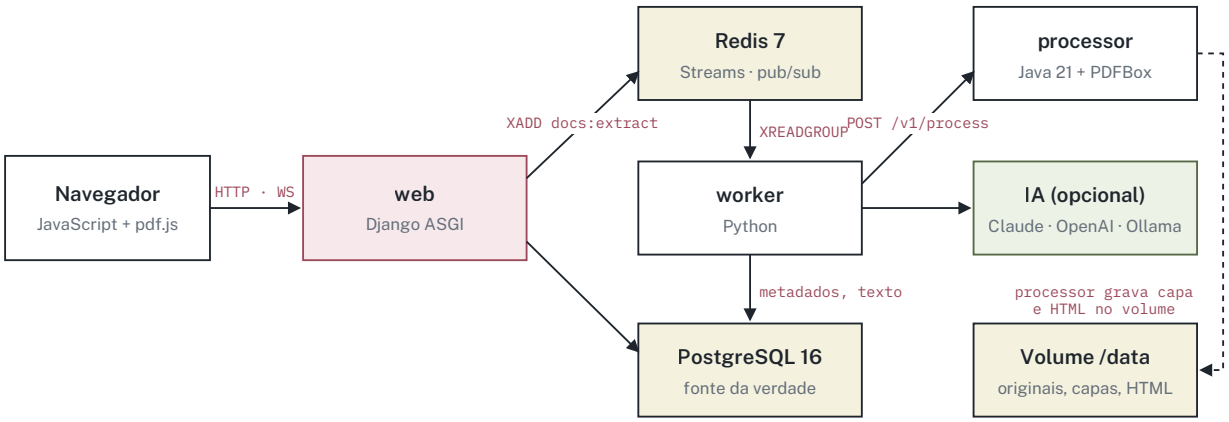


FIG. 3 Detalhe do documento: descrição por IA (em sálvia), etiquetas, estante, coleções e o sumário extraído do arquivo.

04 · ARQUITETURA

Cinco serviços, um caminho

A aplicação web responde rápido e nunca lê o arquivo ela mesma: o trabalho pesado vai para uma fila, é feito por workers e por um processador Java isolado, e o resultado volta ao navegador por WebSocket.



SERVIÇO	TECNOLOGIA	RESPONSABILIDADE
WEB	**Django 5.2, DRF, Channels, Uvicorn**	Páginas, API REST, WebSocket, sessões, permissões e entrega dos arquivos com suporte a Range.
WORKER	**Python**	Consome as filas, chama o processador, grava metadados e texto, escreve a descrição e refaz o índice de busca.
PROCESSOR	**Java 21, PDFBox 3, StAX**	Lê PDF e DOCX: páginas, palavras, sumário, metadados, texto, capa e HTML de leitura. Não acessa banco nem Redis.
POSTGRES	**PostgreSQL 16**	Documentos, leitura, anotações, coleções, contas e histórico do processamento.
REDIS	**Redis 7**	Filas, retentativas, eventos em tempo real, cache e sessões.

05 · DECISÕES TÉCNICAS

Por que foi feito assim

Cada escolha abaixo responde a um problema concreto que apareceu ao transformar "subir um PDF" em algo confiável com vários usuários, vários workers e arquivos imprevisíveis.

01

Um processador Java separado, sem acesso a banco

PDF e DOCX reais vêm quebrados, enormes ou com senha. Isolar a leitura num serviço Java com PDFBox, que só recebe um caminho e devolve JSON, impede que um arquivo ruim derrube a aplicação web.

EFEITO O processador escala sozinho e um `PathGuard` restringe a leitura à pasta de dados.

02

Redis Streams com grupo de consumidores

Uma fila simples perde a tarefa se o worker cair no meio. Com Streams, cada mensagem fica pendente até ser confirmada, e uma varredura recoloca na fila o que ficou para trás — inclusive se o Redis for apagado.

EFEITO `docker compose up --scale worker=3` funciona sem mudar uma linha.

03

O banco decide quem processa: `SELECT … FOR UPDATE SKIP LOCKED`

Mensagens repetidas são normais em filas distribuídas. Antes de trabalhar, o worker trava a linha do documento; se outro já a pegou, ele segue em frente.

EFEITO Nenhum documento é processado duas vezes, mesmo com entrega duplicada.

04

Retentativas com espera exponencial e falha explicada

Falhas passageiras voltam para a fila em 5 s, 15 s, 45 s… Arquivos que nunca vão abrir (com senha, corrompidos, `.doc` antigo) param em "Falhou" com o motivo em português.

EFEITO A pessoa sabe o que fazer; o painel Sistema mostra o histórico de cada etapa.

05

Conteúdo conferido, arquivo deduplicado

A extensão não basta: o `web` confere os bytes do arquivo e calcula o SHA-256 para não guardar o mesmo documento duas vezes. O HTML gerado a partir do DOCX é higienizado com `nh3` antes de chegar ao navegador.

EFEITO Menos disco, e nenhum script embutido num DOCX chega ao leitor.

06

IA opcional, com resumo local de reserva

Exigir chave de API afastaria quem só quer testar. O padrão é um resumo extrativo feito no próprio worker; Claude, OpenAI ou Ollama entram por variável de ambiente, recebendo só título, autor, sumário e um trecho limitado do texto.

EFEITO Funciona offline e envia o mínimo de dados ao provedor.

07

Busca no PostgreSQL, sem serviço extra

`SearchVector` e `SearchRank` para o texto completo, `unaccent` para ignorar acentos e `TrigramSimilarity` para tolerar erros de digitação no título.

EFEITO Busca relevante sem Elasticsearch para manter.

08

Front-end em módulos ES, sem etapa de build

Para um projeto auto-hospedado, cada ferramenta a mais é um ponto de quebra. A interface é JavaScript puro em módulos, com CSS em tokens e fontes auto-hospedadas.

EFEITO Nenhuma chamada a serviços externos e nada para compilar antes de rodar.

06 · DESIGN

Uma ficha de catálogo, não um painel

A identidade vem das bibliotecas: papel creme, tinta escura, etiquetas coloridas e metadados em letra de máquina. Cada cor tem um papel, e o Rosé tinta é a assinatura.

Creme
#F4F1DE
A "mesa": barra lateral

Tinta
#1E252C
Texto e ação principal

Rosé tinta
#9C4A5B
Navegação, progresso, foco

Sálvia escura
#556B47
Tudo o que vem da IA

Ardósia
#8395A7
Estante "lendo"

TÍTULOS E LEITURA

Literata

Serifa feita para leitura longa em tela.

INTERFACE

Public Sans

Neutra e legível em tamanhos pequenos.

METADADOS

IBM Plex Mono

`Contadores e rótulos, como numa ficha.`

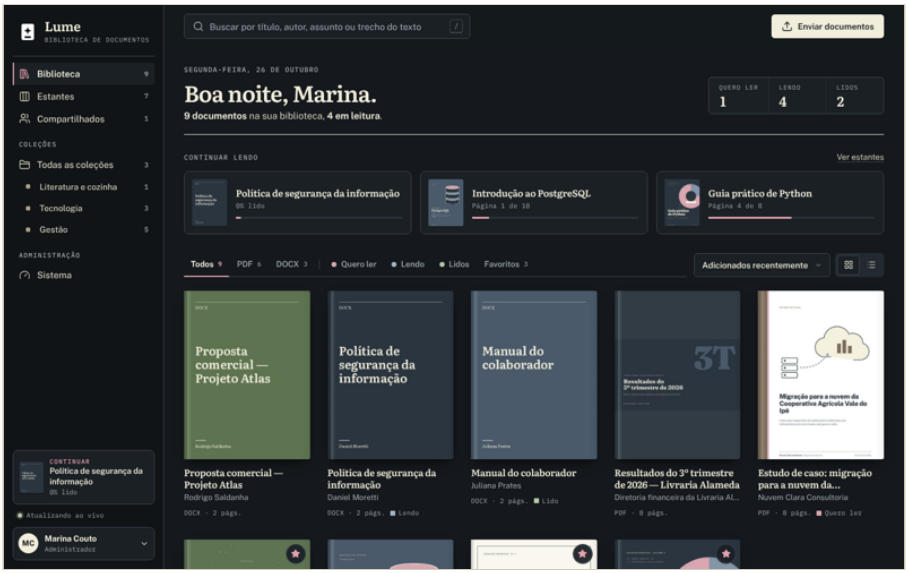


FIG. 4 Tema escuro: mesma hierarquia, assinatura em rosé claro.



FIG. 5 No celular.

Acessibilidade desde o começo

Link "Pular para o conteúdo", anel de foco visível em todos os controles, rótulos ARIA nas ações só com ícone, contraste AA nos dois temas, animações desligadas com `prefers-reduced-motion` e atalhos de teclado na biblioteca e no leitor (`/` busca, `U` envio, `F` modo foco).

07 · TELAS

O resto da casa

Além da biblioteca e do leitor, o Lume tem estantes, busca, envio com progresso ao vivo e um painel para quem administra a instalação.

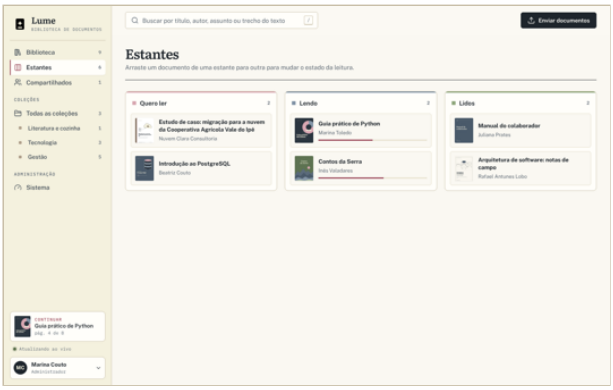


FIG. 6 Estantes: arraste um cartão para mudar o estado de leitura.

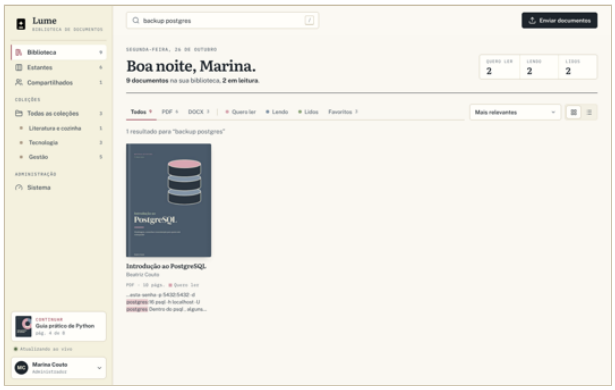


FIG. 7 Busca no texto completo, com o trecho encontrado em destaque.



FIG. 8 Envio: solte os arquivos em qualquer lugar da janela.



FIG. 9 Leitor de DOCX no tema sépia, com sumário pelos títulos do arquivo.

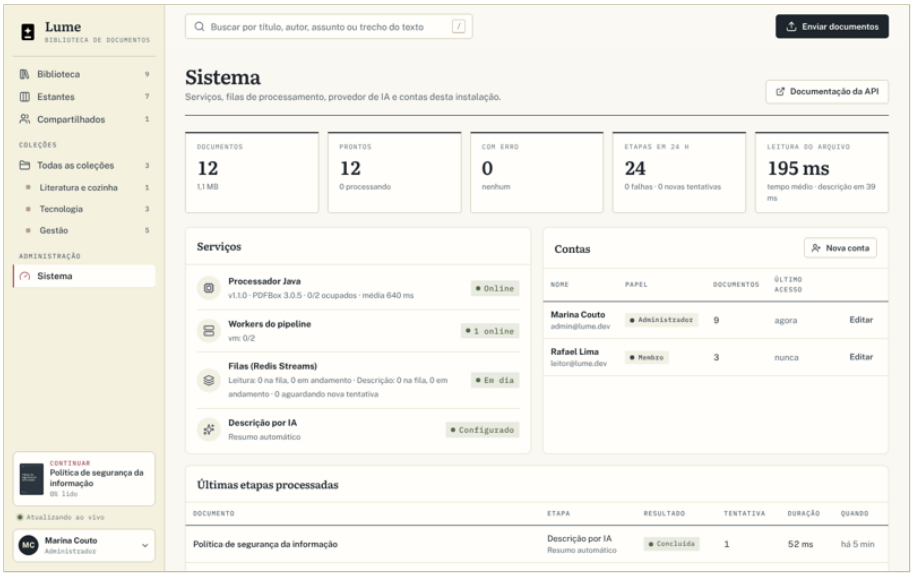


FIG. 10 Sistema: saúde do processador, workers, filas, provedor de IA e contas.

08 · QUALIDADE E ENTREGA

Testado de ponta a ponta

Os testes usam PostgreSQL e Redis de verdade, e uma parte deles passa pelo processador Java com arquivos reais. Nada é simulado onde o erro costuma morar.

PYTEST · 70 TESTES

Do upload ao WebSocket

API, envio, permissões, busca, pipeline, IA, tempo real e painel Sistema. Os de ponta a ponta usam PDF com marcadores, DOCX com títulos e um arquivo corrompido.

GITHUB ACTIONS

Integração contínua

A cada push: compila o processador com Maven, roda todos os testes e constrói as imagens Docker.

UM COMANDO

Rodar em minutos

`iniciar.cmd` ou `iniciar.sh` criam o `.env` com senhas aleatórias, sobem os cinco contêineres e abrem o navegador.

DOCUMENTAÇÃO

Três PDFs, mesma identidade

Manual técnico, Banco de Dados e esta apresentação, gerados a partir de HTML e CSS em `docs/fonte` a cada versão.

Pilha completa

CAMADA	TECNOLOGIA	USO
BACK-END	**Python 3.13 · Django 5.2**	DRF para a API, Channels para WebSocket, Uvicorn como servidor ASGI
PROCESSAMENTO	**Java 21 · PDFBox 3 · StAX**	Texto, sumário, metadados, capas e HTML de leitura
FRONT-END	**JavaScript (módulos ES) · pdf.js 5**	Biblioteca, leitor, estantes, coleções, sistema e conta
DADOS	**PostgreSQL 16 · Redis 7**	Fonte da verdade, busca em texto completo, filas e eventos
INFRAESTRUTURA	**Docker Compose · GitHub Actions**	Cinco serviços, perfil opcional com Ollama, CI a cada push
QUALIDADE	**Pytest · WeasyPrint**	Testes com serviços reais; documentação em PDF

09 · VERSÕES E CRÉDITOS

De onde veio, para onde vai

O Lume nasceu da vontade de ter um lugar único para organizar, ler e anotar documentos, sem depender de vários aplicativos diferentes.

1.1.0 OUTUBRO DE 2026 · REVISÃO DE DESIGN

- Identidade mais autoral: o Rosé tinta como cor de assinatura e IBM Plex Mono para metadados.
- Cabeçalho com números reais, filtros como abas de índice e capas com lombada.
- Acessibilidade: foco visível, contraste AA e respeito a `prefers-reduced-motion`.
- Manuais em PDF reescritos, com a fonte em HTML no repositório.

1.0.0 LANÇAMENTO

- Biblioteca, leitor de PDF e DOCX, descrição por IA, estantes, coleções, busca, tempo real e API REST.

Também para Windows

O **Lume para Windows** (`havelroot/Document-Library-Desktop`) leva a mesma biblioteca para um único `Lume.exe`, sem Docker nem servidor: Electron, SQLite com FTS5, leitura dos arquivos em JavaScript e chaves de IA guardadas no cofre de credenciais do Windows.

Feito a quatro mãos

AUTORIA

Clara A. Portes

Repositório oficial em `Violet-pixel/Document-Library-Platform`.

PARCERIA

Erik Pimenta

Parceiro de projetos e de vida (`@ravelroot`), das decisões de arquitetura ao design do leitor.

Sugestões, issues e pull requests são bem-vindos. Para instalar e operar, veja o Manual técnico; para o modelo de dados, o documento Banco de Dados.

Composto em Literata, Public Sans e IBM Plex Mono (SIL Open Font License 1.1). Gerado com WeasyPrint a partir de `docs/fonte/apresentacao.html`. Lume é distribuído sob a licença MIT · Copyright © 2026 Clara A. Portes.